HEALTH CARE DOMAIN

Saishree Santosh Shet-4SF24IS088

A1.Issues in Given Schema

Initially, I observed that all the data is stored in a single table which mixes patient, doctor, medicine and billing details.I noticed that doctor_name, doctor_specialization and medicine_name are repeated for every record, which creates redundancy and also multiple medicines are stored in the same row,which is not atomic data.Some attributes like medicine_price depend only on medicine_name and not on the full record, which is causing partial dependency.Similarly, bill_amount depends on treatment and medicines, not directly on patient_id, which shows transitive dependency.Because of this schema, it will be difficult to handle multiple doctors, multiple diagnoses or multiple medicines properly.

A2.1NF

To convert into 1NF, I ensured that each column contains atomic values.Initially, medicines were stored in the same row. So I separated medicines into a different table so that each row contains only one value.

Tables in 1NF

1)PatientRecord:

record_id ->Primary Key

patient_id
patient_name
patient_phone
doctor_name
doctor_specialization
appointment_date
diagnosis
treatment
prescription_id
bill_amount
payment_status

2)Medicines

medicine_id ->Primary Key
prescription_id-> Foreign Key
medicine_name
medicine_price

2NF

In 2NF, I removed partial dependencies.I observed that patient, doctor, and medicine details depend on their own , so I separated them into different tables.

Tables in 2NF

1) Patient:

patient_id ->Primary key
patient_name
patient_phone

2)Department:

department_id ->Primary key
department_name

3)Doctor:

doctor_id ->Primary key

doctor_name
doctor_specialization
department_id ->Foreign key

4)Medicine:

medicine_id ->Primary key
medicine_name
medicine_price

5)Consultation:

consultation_id ->Primary key

6)Diagnosis:

diagnosis_id ->Primary key
consultation_id ->Foreign key

7)Prescription:

prescription_id ->Primary key
consultation_id ->Foreign key

8)Prescribed medicine:

prescription_id

medicine_id

These are composite keys..also foregin keys.

Composite Key: to uniquely identify a row using multiple columns.
Foreign Key:to link one table with another table.

9)Bill:

bill_id->Primary key
bill_amount

consultation_id ->Foreign key

10)Payment:

payment_id ->Primary key
payment_status

bill_id->Foreign key

3NF

In 3NF, I made sure there are no indirect dependencies.I checked each table to see if any column depends on something other than the primary key, and I fixed those by separating them.Most of my 2NF design was already fine, so only minor changes were needed.

I updated the Consultation table by adding patient_id as a foreign key so that each consultation is linked to a patient.

1)Consultation:

consultation_id -> Primary Key

patient_id -> Foreign Key

2)I added a new Appointment table to store appointment details separately.

Appointment:
appointment_id -> Primary Key
patient_id -> Foreign Key
doctor_id -> Foreign Key
appointment_date

3)I added diagnosis_description to store actual diagnosis details instead of just IDs.

Diagnosis:

diagnosis_id -> Primary Key

consultation_id -> Foreign Key

diagnosis_description

4)I ensured that indirect dependencies are removed by separating bill and payment details.

Bill:

bill_id -> Primary Key

consultation_id -> Foreign Key

bill_amount

Payment:

payment_id -> Primary Key

bill_id -> Foreign Key

payment_status

A3.Assumptions i made:

While redesigning the schema, I made some basic assumptions based on how I think a hospital system works. I assumed that each patient will have a unique patient_id and all their details are stored using that. A patient can visit the hospital multiple times, so one patient can have many consultations. I considered that each consultation stores medical details like diagnosis and treatment separately. Also, one consultation can have multiple diagnoses, so I handled that using a separate table. Medicine details like name and price are stored separately since they can be reused for different patients. I also assumed that each doctor belongs to one department, but a department can have

multiple doctors. Billing is generated for each consultation and includes the total cost, and payment status is maintained separately for each bill,also consultation can involve multiple doctors, so I thought of using a separate table.

Part B:

B1:Requirement Clarification:

Before starting the design, I had some doubts while understanding the system, so I noted a few questions.I was thinking whether a patient can book multiple appointments on the same day with different doctors. I was also not sure if one consultation can involve more than one doctor.I also thought about whether a single consultation can have multiple diagnoses and treatments.For prescriptions, I was confused if we need to store details like dosage and duration for each medicine.Lastly, I had a doubt about billing, whether it is generated per consultation or separately for each treatment.

B2:Database design:

Based on the given requirements, I designed the database by separating different entities like patient, doctor, consultation, and billing into different tables. This helps in reducing redundancy and maintaining proper structure.

From the requirements, I identified the main entities as Patient, Doctor, Department, Appointment, Consultation, Diagnosis, Prescription, Medicine, Bill and Payment.

Tables:

1)Patient:I created a Patient table to store patient details separately so that the same patient data is not repeated.

Patient:

patient_id -> Primary Key

patient_name

patient_phone

2)Department:Doctors belong to departments, so I created a Department table.

Department:

department_id -> Primary Key

department_name

3)Doctor:I created a Doctor table to store doctor details and linked it with department.

doctor_id -> Primary Key

doctor_name

doctor_specialization

department_id -> Foreign Key

4)Appointments:Appointments store the interaction between patient and doctor with date.

appointment_id -> Primary Key
patient_id -> Foreign Key
doctor_id -> Foreign Key
appointment_date

5)Consultation:Consultation stores medical details for each visit.

consultation_id -> Primary Key
patient_id -> Foreign Key
appointment_id -> Foreign Key

6)Prescription:Prescription is created for each consultation.

prescription_id -> Primary Key
consultation_id -> Foreign Key

7)Medicine:Medicine details are stored separately for reuse.

medicine_id -> Primary Key
medicine_name
medicine_price

8)Prescribed Medicine:Since one prescription can contain multiple medicines, I created a mapping table.

prescription_id
medicine_id
Primary Key:
(prescription_id, medicine_id)

10)Diagnosis:

diagnosis_id -> Primary Key

consultation_id -> Foreign Key

diagnosis_description

11)Consultation_Doctor:

consultation_id

doctor_id

contribution

Primary Key:

consultation_id, doctor_id

12)Bill:Billing is maintained for each consultation.

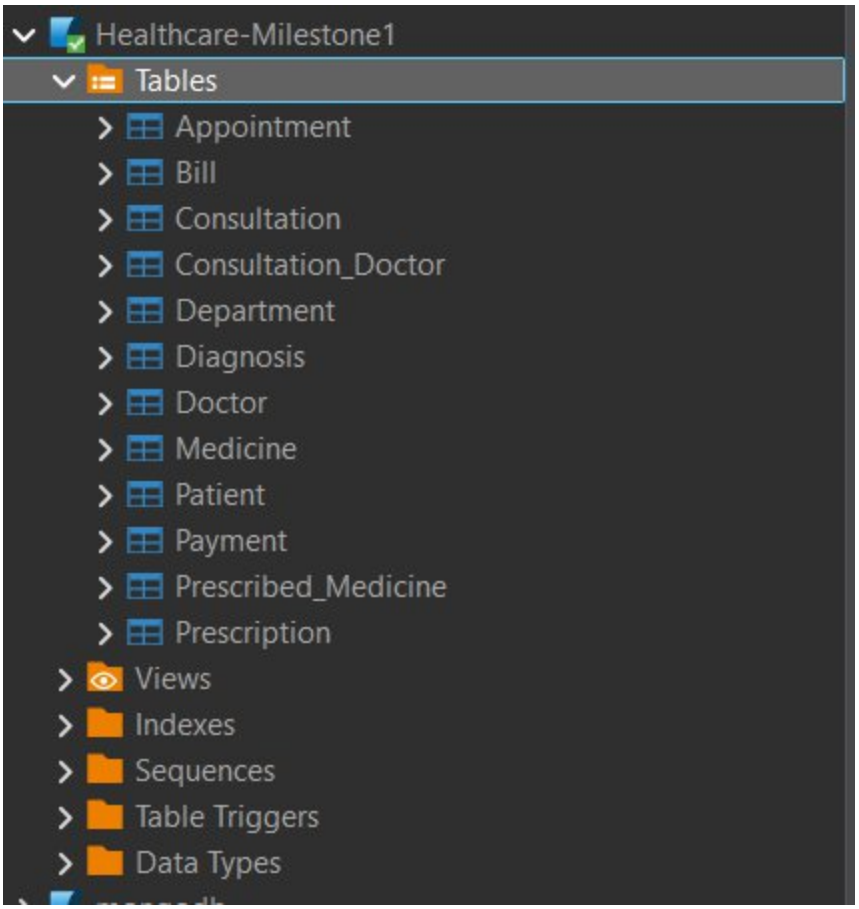
bill_id -> Primary Key
consultation_id -> Foreign Key
bill_amount

13)Payment:Payment status is tracked separately.

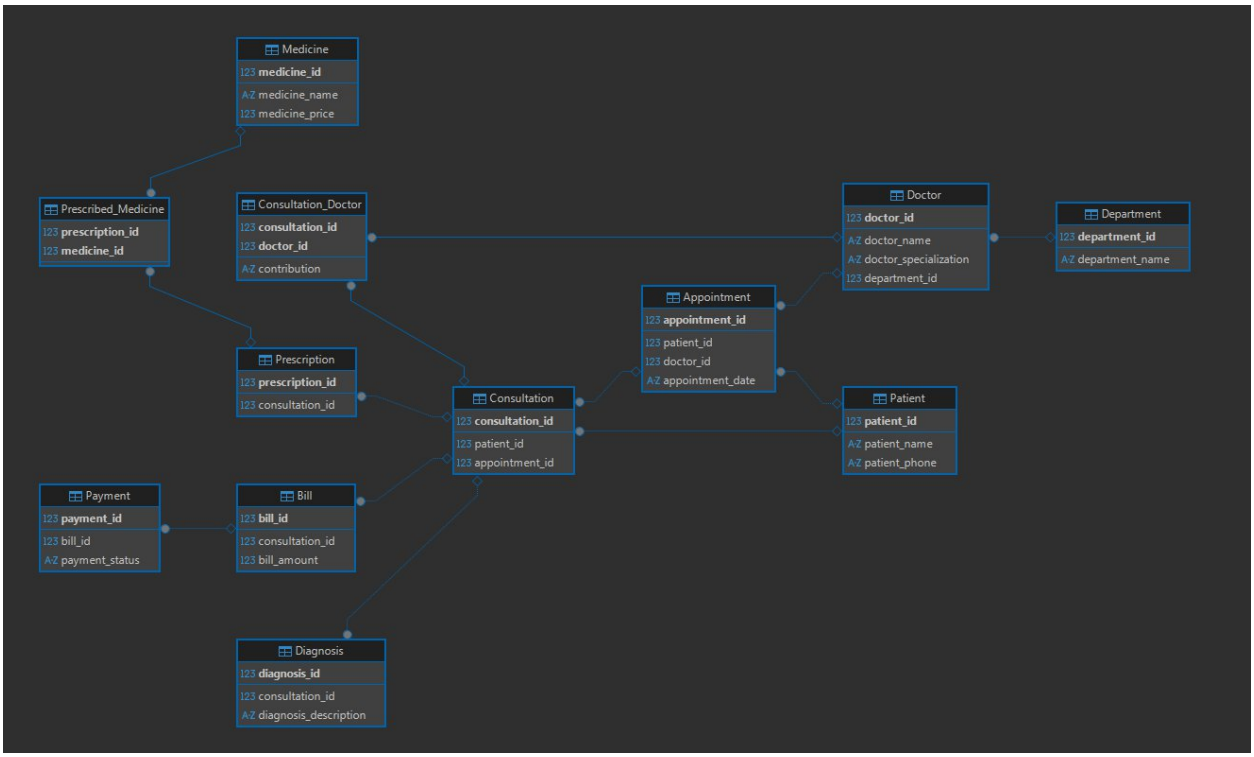
payment_id -> Primary Key
bill_id -> Foreign Key
payment_status

Relationships:

Patient → Appointment (one-to-many)
Doctor → Appointment (one-to-many)
Consultation → Diagnosis (one-to-many)
Consultation → Prescription (one-to-many)
Prescription → Medicine (many-to-many)
Consultation → Doctor (many-to-many)



ER-Diagram:The ER diagram below represents the relationships between all tables.



The schema was implemented and verified using DBeaver.

B3:Normalization:

The designed schema is normalized up to Third Normal Form (3NF).

All tables have atomic attributes (1NF).

There are no partial dependencies as data is separated into different tables (2NF)

There are no transitive dependencies, and all non-key attributes depend only on the primary key (3NF)

Update during development:

Initially, I assumed that one consultation involves only one doctor.After the requirement change, I modified the design to support multiple doctors per consultation.

To handle this, I created a separate table:

Consultation_Doctor:
consultation_id
doctor_id
contribution

Primary Key:
consultation_id, doctor_id

Foreignkeys are:consultation_id,doctor_id

This update makes the schema flexible and suitable for real-world scenarios where multiple doctors collaborate.

Section C:Queries and Transactions

C1. Consultation History of a Patient:This query retrieves the complete consultation history of a patient, including the doctors consulted, diagnosis details, and prescribed medicines.

```
SELECT

    p.patient_name,

    c.consultation_id,

    d.doctor_name,

    diag.diagnosis_description,

    m.medicine_name

FROM Patient p

JOIN Consultation c ON p.patient_id = c.patient_id

JOIN Consultation_Doctor cd ON c.consultation_id = cd.consultation_id

JOIN Doctor d ON cd.doctor_id = d.doctor_id

LEFT JOIN Diagnosis diag ON c.consultation_id = diag.consultation_id

LEFT JOIN Prescription pr ON c.consultation_id = pr.consultation_id

LEFT JOIN Prescribed_Medicine pm ON pr.prescription_id = pm.prescription_id

LEFT JOIN Medicine m ON pm.medicine_id = m.medicine_id;
```

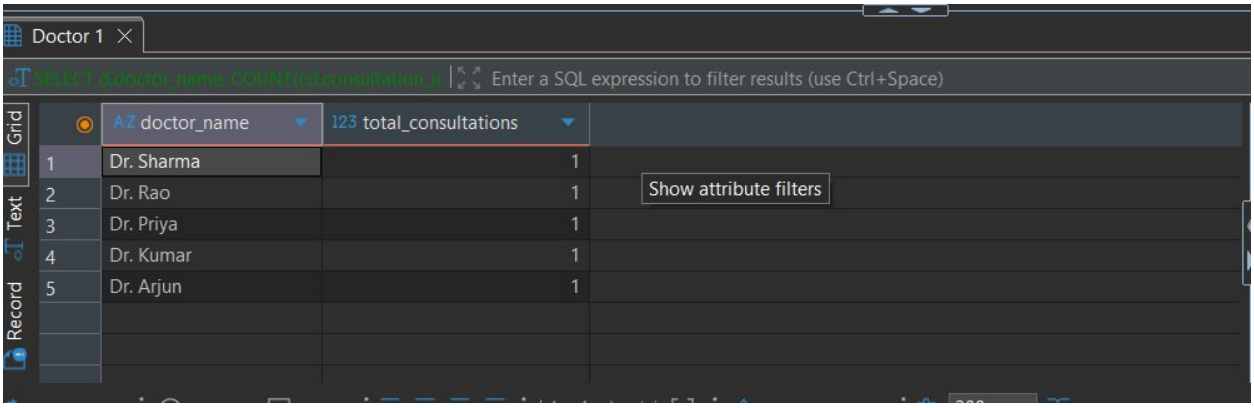
Patient(+) 1

Enter a SQL expression to filter results (use Ctrl+Space)

	patient_name	consultation_id	doctor_name	diagnosis_description	medicine_nam
1	Sai	1	Dr. Rao	Fever	Paracetamol
2	Anu	2	Dr. Sharma	Migraine	Ibuprofen
3	Ravi	3	Dr. Kumar	Fracture	Calcium
4	Meena	4	Dr. Priya	Skin Allergy	Antihistamine
5	Kiran	5	Dr. Arjun	Cold	Cough Syrup

C2. Doctor with Highest Consultations:This query calculates the number of consultations handled by each doctor and displays them in descending order to identify the doctor with the highest consultations.

```
SELECT

    d.doctor_name,

    COUNT(cd.consultation_id) AS total_consultations

FROM Doctor d

JOIN Consultation_Doctor cd

ON d.doctor_id = cd.doctor_id

GROUP BY d.doctor_name

ORDER BY total_consultations DESC;
```

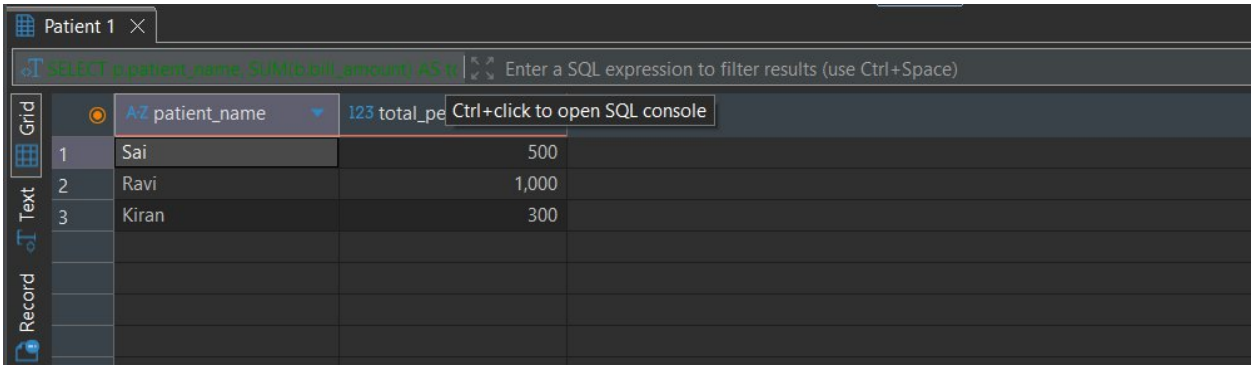


C3. Patients with Pending Payments:This query lists all patients who have pending payments along with the total outstanding bill amount.

```
SELECT

    p.patient_name,

    SUM(b.bill_amount) AS total_pending

FROM Patient p

JOIN Consultation c ON p.patient_id = c.patient_id

JOIN Bill b ON c.consultation_id = b.consultation_id

JOIN Payment pay ON b.bill_id = pay.bill_id
```

WHERE pay.payment_status = 'Pending'

GROUP BY p.patient_id, p.patient_name;



C4. Transaction:This transaction handles the complete process of recording a consultation, adding diagnosis, prescribing medicines, generating a bill, and recording payment. All operations are executed together to maintain consistency.

BEGIN TRANSACTION;

INSERT INTO Consultation VALUES (6, 1, 1);

INSERT INTO Diagnosis VALUES (6, 6, 'Headache');

INSERT INTO Prescription VALUES (6, 6);

INSERT INTO Prescribed_Medicine VALUES (6, 1);

INSERT INTO Bill VALUES (6, 6, 600);

INSERT INTO Payment VALUES (6, 6, 'Pending');

COMMIT;

If any step fails, the transaction can be rolled back to maintain data consistency.

SELECT * FROM Consultation;

Consultation 1

	consultation_id	patient_id	appointment_id
1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4
5	5	5	5

SELECT * FROM Diagnosis;

Diagnosis 1

	diagnosis_id	consultation_id	diagnosis_description
1	1	1	Fever
2	2	2	Migraine
3	3	3	Fracture
4	4	4	Skin Allergy
5	5	5	Cold

SELECT*FROM Bill;

Bill 1

	bill_id	consultation_id	bill_amount
1	1	1	500
2	2	2	700
3	3	3	1,000
4	4	4	400
5	5	5	300

In this section, I used SQL queries with joins and aggregation to retrieve and analyze data from the database. I also implemented a transaction to ensure consistency while performing multiple operations. These queries show how the database can be used effectively in real-world scenarios.